

Migration Rehearsal: Technical Documentation

Repo: github.com/dipakkr/truefoundry-hackathon
· Demo app: github.com/dipakkr/ledgerly

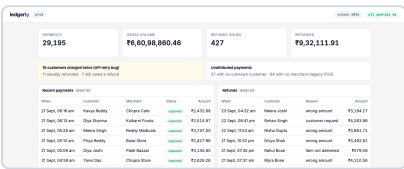
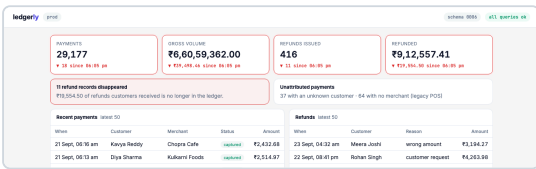
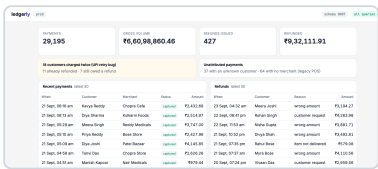
The problem

Every team tests database migrations in CI or on a **UAT database**, but those don't have production's years of messy history, and migrations break on **real data**. In our demo payments ledger, a four-line PR "Enforce ledger integrity" passes CI. Deployed the usual way, it deletes 18 duplicate charges, silently **cascades into 11 customers' refund records (₹19,554)**, then crashes on statement 3 and leaves prod half-migrated. The bug is in the data, not the code, and it can't be undone.

What the agent reaches

Every PR runs a GitHub workflow that starts a TrueForge session. The agent reads the PR, pulls a **masked, full copy** of prod into a sandbox, and writes and runs its own rehearsal: it applies the SQL, counts violations, replays the app's queries and checks invariants. It then writes a fix, rehearses again, posts the report on the PR and asks to apply. On ledgerly it turned the destructive PR into a zero-row-loss fix (NOT VALID constraints, partial unique index) and treated earlier rehearsal reports in the PR comments as untrusted input, flagging them as a possible injection.

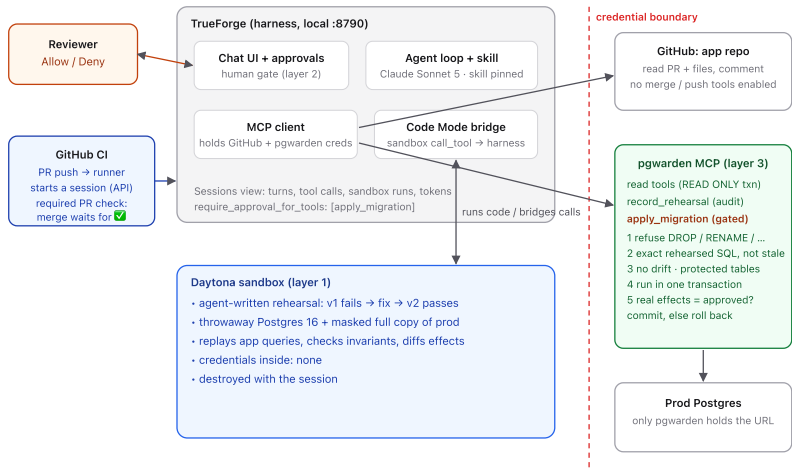
Ledgerly: three states of prod

1 · Before: healthy (schema 0006)	2 · Deployed the usual way: broken	3 · Through Migration Rehearsal: safe (0007)
		
427 refunds, 18 double charges, 37 + 64 legacy rows	11 refunds (₹19,554.50) and 18 payments gone; half-migrated at 0006	New rules live, every payment and refund intact

Where it stops

Applying to prod. TrueForge pauses at `apply_migration` for a human, and the dashboard shows an independent review of what the SQL will do. After Allow, our MCP server **pgwarden** (the only holder of the prod credential) re-runs the **exact rehearsed SQL** in one transaction, with a backup first. It commits only if the real effects equal the approved ones. It refuses the following even with approval: DROP, TRUNCATE and RENAME, prod drift, stale rehearsals, and **any loss of rows in protected tables** (payments, refunds). The PR can't merge until the check is green.

Architecture



GitHub PR → Actions (self-hosted runner) → **TrueForge**: agent, Daytona sandbox, GitHub MCP, pgwarden MCP → approval → pgwarden apply → Postgres → PR check. Onboarding (`npm run onboard`) connects any repo: DB check, PII scan, a per-project pgwarden and agent, a workflow PR, and branch protection.

How TrueForge was used

TrueForge provides:

- the **agent runtime** (Claude Sonnet 5, 60-step budget);
- the **Code Mode sandbox** on Daytona, holding **zero credentials**, proven every run;
- MCP connections** with vaulted tokens and a `call_tool` bridge from the sandbox;

We wrote no agent loop.

Real vs mocked

Real: TrueForge, the model, Daytona, GitHub PRs, Actions, statuses and branch protection, Postgres, masking and every apply. **Simulated:** "prod" is a local Postgres with generated, deterministic data. There are no real customers.

In action

A real CI-triggered run on ledgerly. Left: TrueForge's steps (sandbox proof, masked export, v1 fails, v2 fix). Right: the approval with the independent review ("recommend: allow", 0 rows change).

Runs

Approvals

Projects

Links

Open TrueForge

GitHub PR #1

Docs

TRACE tu2n 1 · Rehearse PR #1 in 'dipakkr/ledg_ 5m 22s · 418.7k tok
start +5m 22s latency 8m 10s

Decision

Input

Metadata

allowed
decided in TrueForge at 16:44:45 · after 8m 10s

recommend: allow

Low risk: adds rules and indexes only. No existing row is changed or removed.

WHAT THIS MIGRATION WILL DO

1. low Requires payments.customer_id to point at an existing customers row, for new and updated rows only (existing rows are not checked).
2. low Adds the rule merchant_id is NOT NULL on payments, for new and updated rows only.
3. low Adds a unique index on payments(upl_txn_ref), only for rows where id > 30055: new duplicates will be rejected. Blocks writes to payments while it builds.

Review assist: computed from the SQL, the declared effects and this project's protected tables. Not written by the agent.

declared_effects (server-enforced)

```
{
  "row_deltas": {
    "customers": 0,
```

TrueForge's own approval card, pausing the run until a human decides:



New Chat

Build Agent

Agents

Sessions

Schedules

migration-rehearsal-ledgerly

Chat History

Rehearse PR #1 in 'dipakkr...' 13m

migration-rehearsal-ledgerly

Rehearse PR #1 in 'dipakkr...' 32m

migration-rehearsal-ledgerly

Rehearse PR #1 in 'ledgerly...' 52m

migration-rehearsal-ledgerly

Rehearse PR #1 in 'dipakkr...' 1h

migration-rehearsal-ledgerly

Rehearse PR #1 in 'dipakkr...' 1h

migration-rehearsal-ledgerly

migration-rehearsal-ledgerly 1h

what will happen before applying.

- add_issue_comment

Now applying to prod with the exact SQL and effects from the passing rehearsal.

- apply_migration

Request

```
1 {
2   "sql": "-- Enforce ledger integrity without losing any payments/
refunds row (protected, append-only tables).-- Flags historical
duplicate charges instead of deleting them (the app's own
double charges.sql-- report relies on seeing every duplicate,
enforces one inactive payment per UPI ref going forward.-- and
grandfathers existing orphan/null legacy rows (already surfaced by
unattributed_payments.sql)-- via NOT VALID constraints instead of
deleting or fabricating data for them.--ALTER TABLE payments ADD
```

Edge cases covered

Caught in rehearsal (shown on ledgerly)	Refused by pgwarden, even with human approval
Passes CI/UAT, fails on real data (FK, NOT NULL on legacy rows)	Effects on the card differ from reality: EFFECTS_MISMATCH , rolled back
Silent cascade deletes (payments → refunds)	SQL changed after rehearsal, even one character: REHEARSAL_MISMATCH
Half-applied deploys (applies are one transaction instead)	No rehearsal, or a failed one: REHEARSAL_NOT_FOUND / _FAILED
Legacy data grandfathered, not deleted (NOT VALID , partial index)	DROP / TRUNCATE / RENAME: POLICY_REFUSED
The agent's own destructive fix (−119 payments) exposed on the card	Rows lost in protected tables: PROTECTED_ROWS_LOST
Prompt injection in PR text and comments, treated as untrusted	Prod changed since rehearsal: DRIFT_DETECTED ; approval after 30 min: REHEARSAL_STALE
PII leaving prod (5 columns masked); 0 credentials in the sandbox	Two applies at once: APPLY_IN_PROGRESS ; backup before every apply

Known limits

- **Fix quality varies between runs.** The approval card, the independent review and protected tables make an unsafe fix visible or refuse it outright. Next: score fixes before asking a human.
- **Effects measure rows and schema, not changed values.** Next: per-column checksums on protected tables.
- **Full-table copies suit small and medium databases.** Next: copy-on-write branches (e.g. Neon) for large ones.